

Guia de Testes — HybriD-GM (simulador_quantico)

Como rodar a suíte de testes, o que cada camada verifica, e como os parâmetros de linha de comando dos três executáveis (general.out / grover.out / shor.out) se encaixam — incluindo as pegadinhas mais comuns.

1. Como rodar tudo de uma vez

Nesta máquina existem dois caminhos de build, escolhidos pela variável GPU do makefile:

Windows nativo, sem GPU real (GPU=stub) — rápido, para iteração no dia a dia

```
mingw32-make GPU=stub test
```

Usa src/core/kernel_stub.cpp no lugar do kernel CUDA de verdade. Os backends t_GPU/t_HYBRID rodam, mas só imprimem um aviso e não calculam nada real — servem só para confirmar que o programa não trava/crasha com esses parâmetros.

WSL2 + CUDA real (GPU=real) — mais lento de compilar, mas testa a GPU de verdade

```
make GPU=real test
```

Usa o kernel.cu de verdade (nvcc). Necessário sempre que a mudança envolve t_GPU/t_HYBRID de fato, região/coalescimento de GPU, ou performance.

Atenção (visto nesta máquina): o build Windows nativo pode falhar ao linkar os binários maiores (collect2.exe: error: ld returned 116 exit status, arquivo de 0 bytes) por interferência do antivírus (Windows Defender) — não é um problema do código. Se acontecer, use o caminho WSL/GPU=real acima, ou adicione uma exclusão do Defender para a pasta do MSYS2 e/ou outputs/ do projeto.

Só rodar o smoke test de novo, sem recompilar

```
bash tests/smoke_test.sh
```

2. As três camadas de teste

Camada	O que verifica	Usa GPU real?
test_qft_addf.out	Regressão de string: compara a saída de circuitos de soma/QFT contra um resultado gravado. Não passa por DGM::execute().	Não
test_gates.out	Regressão de execução: roda CNot / Toffoli / ControlledN / Controlled1 via DGM::execute() (backend t_CPU), 32 casos de tabela-verdade.	Não
tests/smoke_test.sh	Ponta a ponta: chama general.out / grover.out / shor.out já buildados, com poucos qubits, e confere o resultado (amplitude exata, ou taxa de sucesso ao longo de várias execuções para os algoritmos probabilísticos).	Sim, se GPU=real

3. Os três binários e seus argumentos

Os três seguem o mesmo esqueleto de argumentos posicionais:

```
outputs/general.out <qubits> <exec_type> [thread_count|gpu_count] [block_size] [repeat_count] [gpu_region_bi
outputs/grover.out <qubits> [exec_type] [thread_count|gpu_count] [block_size] [repeat_count] [gpu_region_bi
```

outputs/shor.out <qubits> [exec_type] [thread_count|gpu_count] [block_size] [repeat_count] [gpu_region_bits]

general.out exige `exec_type ≥ 1` (não aceita `t_CPU`). grover.out e shor.out aceitam os 4 valores, com default `t_CPU` se omitido.

exec_type

Valor	Backend	Descrição
0	t_CPU	Serial, um núcleo só.
1	t_PAR_CPU	CPU paralela via OpenMP.
2	t_GPU	GPU (CUDA), todo o vetor de estado de uma vez.
3	t_HYBRID	CPU + GPU dividindo o trabalho em regiões.

O terceiro argumento MUDA DE SENTIDO conforme exec_type — a pegadinha mais comum

exec_type	3º argumento significa...
1 (t_PAR_CPU) ou 3 (t_HYBRID)	thread_count — quantas threads OpenMP
2 (t_GPU)	gpu_count — quantas GPUs

Se você passar, por exemplo, `t_HYBRID` (3) com um número que fazia sentido como `gpu_count`, ele vai ser lido como `thread_count` — o programa roda e dá uma saída plausível, mas com o parâmetro errado. Não há validação cruzada nessa posição: o código simplesmente confia no `exec_type` para decidir o que o 3º número significa.

block_size / repeat_count / gpu_region_bits (posições 4/5/6)

Só valem para `t_GPU/t_HYBRID` (ignorados, mas inofensivos, nos outros backends). Não são livres — `DGM::validateTuning()` exige a igualdade abaixo antes de qualquer execução:

$2 * \text{block_size} * \text{repeat_count} == 2 ^ \text{gpu_region_bits}$

Se a conta não bater, o programa imprime "Erro de tuning" e sai — isso é validação intencional, não bug. Os defaults (`block_size=64`, `repeat_count=2`, `gpu_region_bits=8`) já satisfazem a igualdade ($2 \cdot 64 \cdot 2 = 256 = 2^8$); mudar um valor exige ajustar os outros dois junto.

search_value (só em grover.out, posição 7)

Precisa caber em `qubits-1` bits. Fora do intervalo, o programa recusa com uma mensagem clara em vez de rodar silenciosamente com um valor truncado.

4. Coisas que parecem bug mas não são

Sintoma observado	Explicação
shor.out imprime "Failed to find factors" em alguma execução	Esperado: Shor é probabilístico, e a taxa de sucesso historicamente é baixa e variável mesmo com tudo funcionando certo. Uma única execução não prova nada — por isso o smoke test roda 16 vezes e só exige 1 sucesso.
t_GPU/t_HYBRID "não fazem nada" / imprimem aviso no stderr	Build está em GPU=stub (kernel_stub.cpp) — sem GPU real por trás. Precisa recompilar com GPU=real (via WSL nesta máquina) para exercitar a GPU.

Sintoma observado	Explicação
"Erro de tuning" ao mudar <code>block_size/repeat_count/gpu_region_bits</code>	Validação intencional (ver seção 3) — os três precisam satisfazer $2 \cdot \text{block_size} \cdot \text{repeat_count} = 2^{\text{gpu_region_bits}}$.
<code>t_HYBRID</code> com resultado difícil de entender / quero ver quem processou o quê	Rode com a variável de ambiente <code>HYBRID_DEBUG=1</code> — imprime, região por região, se foi a CPU (qual thread) ou a GPU que processou, e o total de cada lado no fim.

Exemplo: ligando o log detalhado do modo híbrido

```
HYBRID_DEBUG=1 ./outputs/general.out 24 3 4
```

5. Referência rápida de comandos

Comando	O que faz
<code>outputs/general.out 18 1 4</code>	Hadamard em 18 qubits, <code>t_PAR_CPU</code> , 4 threads.
<code>outputs/general.out 20 2 1</code>	Hadamard em 20 qubits, <code>t_GPU</code> , 1 GPU.
<code>outputs/general.out 22 3 4</code>	Hadamard em 22 qubits, <code>t_HYBRID</code> , 4 threads CPU (+ GPU).
<code>outputs/grover.out 12 0</code>	Grover em 12 qubits, <code>t_CPU</code> (default), <code>search_value</code> default (10).
<code>outputs/grover.out 12 3 4 64 2 8 5</code>	Grover em 12 qubits, <code>t_HYBRID</code> , 4 threads, tuning de GPU default explícito, buscando o valor 5.
<code>outputs/shor.out 15 0</code>	Fatoração de Shor, <code>t_CPU</code> , número-alvo mapeado para 15 qubits (57).
<code>HYBRID_DEBUG=1 outputs/general.out 24 3 4</code>	Mesmo que a 3ª linha, com log detalhado de região por região.

Gerado a partir da conversa sobre a suíte de testes do projeto. Para o histórico completo de bugs encontrados/corrigidos e o desenho do motor de execução, ver `docs/07-bugs-e-pontos-de-atencao.md` e `docs/01-arquitetura-geral.md` no repositório.